

Lab 1

[valid 2024-2025]

Introduction to [Java Programming](#)

1. Compulsory (1p)

Write a Java application that implements the following operations:

- Display on the screen the message "Hello World!". Run the application. If it works, go to step 2 :)
- Define an array of strings `languages`, containing {"C", "C++", "C#", "Python", "Go", "Rust", "JavaScript", "PHP", "Swift", "Java"}
- Generate a random integer n : `int n = (int) (Math.random() * 1_000_000);`
- Compute the result obtained after performing the following calculations:

```
multiply n by 3;
add the binary number 10101 to the result;
add the hexadecimal number FF to the result;
multiply the result by 6;
```

- Compute the sum of the digits in the result obtained in the previous step. This is the new result. While the new result has more than one digit, continue to sum the digits of the result.
- Display on the screen the message: "Willy-nilly, this semester I will learn " + `languages[result]`.

2. Homework (2p)

- Let n, k be two integers given as command line arguments.
- Create a $n \times n$ matrix, representing the adjacency matrix of [random graph](#) G , having the following properties (if it is possible):
 - It contains at least one clique of size k .
 - It contains at least one stable set of size k .
- Implement a method that builds a string representation of the matrix, at your own choice. Display on the screen the generated matrix. You might want to use the *geometric shapes* from the [Unicode chart](#) to create a "pretty" representation of the matrix.
- Display the number of edges m .
- Display the maximum and the minimum degree of a vertex, $\Delta(G)$ and $\delta(G)$ (use the *greek letters*).
- Verify that the sum of the degrees equals the value $2 * m$.
- For larger n display the running time of the application (and do not display the matrix!). Try $n > 30_000$. You might want to adjust the JVM Heap Space using the VM options `-Xms4G -Xmx4G`.
- Launch the application from the command line, for example: `java GraphGenerator 10 4`.

3. Bonus (2p)

- Write an algorithm that finds if a graph has a clique of size at least k . Test you algorithm for randomly generated instances of various sizes.
- Use the algorithm above to check if a graph has a stable set of size at least k .

Notes

- Create a new project for each laboratory. In a project, create at least one package. Create classes as necessary, do not use the same class for unrelated tasks. Each class must contain the name(s) of the author(s) as a comment.

- Please consult [the API documentation](#) to learn more information about the classes and methods used!
- Write code faster using [keyboard shortcuts](#).

Resources

- [Slides](#)
- Download [JDK](#)
- Download [Netbeans](#) or other Java IDE.
- [Language Basics](#)
- [Numbers and Strings](#)
- [NetBeans IDE Java Quick Start Tutorial](#)

Objectives

- Get used with an integrated development environment (IDE): [Netbeans](#).
- Get used with the Java language syntax.
- Create and run a simple application.
- Understand the following concepts: compiler, interpreter, byte-code, Java Virtual Machine (JVM), portability.
- Use the [Unicode](#) alphabet and Java special characters.
- Work with primitive data types and strings ([String](#), [StringBuilder](#)).
- Work with one- and multi-dimensional arrays.
- Parse command line arguments.
- Perform conversions between strings and numbers.
- Split a string into tokens.
- Generate random numbers.
- Create multiple methods in the main class of the application.